

Divide-and Conquer algorithm Report

Name : Medjina Agar Azor

Student ID: 411221413

This report provides the result of an experiment where I was able to execute and compare the performance of Merge Sort and Quick Sort with different thresholds, input sizes, and input data types. The aim of the experiment was to examine the effect of the choice of sorting algorithm and input data on the execution time.

Algorithms Used

Merge Sort:

Merge Sort is a sort algorithm that separates the input array in half, continued until all sub-arrays have one element. Once the array is split into individual elements, it sorts the sub-arrays as a whole. The merge function receives the two halves of the sorted array and merges them into a single complete array, ensuring that the elements are placed into order. It does so recursively on all sub-arrays and then combines as sub-arrays unite, and an entirely sorted array results. Merge Sort follows the divide-and-conquer approach and is known to be the best-performing algorithm and takes up worst-case time of $O(n \log n)$.

Quick Sort:

Quick Sort utilizes the divide-and-conquer approach since the array would split roughly around the pivot. The partitioning places elements less than the pivot on the left and larger than the pivot on the right. This algorithm is repetitively used to sub-arrays, the pivot being in the correct sorted position. The partition is a key operation in this algorithm since it selects the pivot and reorders the array around it. Quick Sort is very efficient in real life and usually executes with a time complexity of $O(n \log n)$ in the average scenario. However, in the worst scenario, if the pivot selection is not good, its time complexity drops down to $O(n^2)$.

Terminal Conditions

I chose to add terminal conditions to transition from recursive algorithms (Merge Sort and Quick Sort) to the non-recursive Insertion Sort when the sub-array size is small (according to the thresholds).

My chosen non-recursive solution is Insertion Sort, which is a simple algorithm that can sort an array one at a time. When the array is small, it is better to use Insertion Sort.

Input Sizes and Testing

I executed the algorithms for three input sizes: 1000, 10000, and 100000. These input sizes were chosen in order to represent small, medium, and large arrays.

Experiment Results

Array Size: 1000, Threshold: 5

Merge Sort Time: 0 ms

Quick Sort Time: 0 ms

Array Size: 10000, Threshold: 5

Merge Sort Time: 2 ms

Quick Sort Time: 1 ms

Array Size: 100000, Threshold: 5

Merge Sort Time: 36 ms

Quick Sort Time: 25 ms

Array Size: 1000, Threshold: 10

Merge Sort Time: 0 ms

Quick Sort Time: 0 ms

Array Size: 10000, Threshold: 10

Merge Sort Time: 3 ms

Quick Sort Time: 1 ms

Array Size: 100000, Threshold: 10

Merge Sort Time: 37 ms

Quick Sort Time: 25 ms

Array Size: 1000, Threshold: 20

Merge Sort Time: 0 ms

Quick Sort Time: 0 ms

Array Size: 10000, Threshold: 20

Merge Sort Time: 2 ms

Quick Sort Time: 1 ms

Array Size: 100000, Threshold: 20

Merge Sort Time: 34 ms

Quick Sort Time: 26 ms

Note: These are the results after running the test twice.

Results Analysis

For small input sizes (1000), both Merge Sort and Quick Sort were so fast that the times were recorded as 0 ms. For bigger inputs, I observed slight but consistent improvements when increasing the threshold. Quick Sort was slightly better in general than Merge Sort for all input sizes, especially as the size got bigger.

As predicted, applying Insertion Sort to small subarrays improved both algorithms' performance. Nevertheless, the variation between thresholds 10 and 20 was insignificant, indicating that past a certain threshold, the threshold increase returns diminishing benefits. These findings conform to my predictions and illustrated the effect of the input type and the selection of the sorting algorithm.

Further Experiments with Varying Input Orders and Comparison with `std::sort()`

Both types (Merge Sort and `std::sort()`) likely had similar performance, with `std::sort()` perhaps performing a hair faster due to its highly optimized implementation. For the small arrays (e.g., 1000 elements), the times were recorded as 0 ms because the algorithms ran so quickly. For the large arrays (e.g., 10000 and 100000), the time differences started to show.

Sorted Array: Sorting a sorted array was very fast for both the algorithms.

Reverse Sorted Array: Sorting a reverse-sorted array was more difficult (especially for Merge Sort), so time was marginally more. The program also checked if the two sorts produced the same output.

Conclusion

In this experiment, I compared the efficiency of Merge Sort and Quick Sort with different thresholds, input sizes, and types of input. The tests showed that both algorithms performed equally well, although Quick Sort was generally a tad faster, especially for large arrays. I learned that the threshold value, utilized to determine whether or not to apply Insertion Sort, played a significant role in performance, especially for small arrays. Both algorithms handled varied types of input—i.e., sorted, reverse-sorted, and random arrays—alike. Overall, the experiment helped me understand the importance of choosing the right sorting algorithm and threshold based on the size and type of the input and provided me with some valuable insights into each algorithm's capabilities and limitations.